Introduction to Security

OSTEP 53

Sam Ogden

Updated 2026-04-28

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

Lecture Objectives

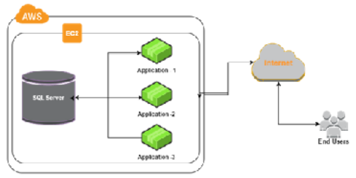
- After this lecture, you should be able to:
- Explain why security is an essential consideration for operating systems
- Discuss the goals of security
- Core security concepts

What we’ve done so far

- So far we’ve virtualized resources:
  - Limited Direct Execution for processes
  - Paging for memory
- That keeps processes apart from each other
- But isolation alone does not answer how shared resources should be controlled
- Is that really enough?

Resource Sharing is still important

- Not all applications can share memory with just threads
- Sometimes we need to communicate between processes



Many servers run databases in separate processes

What might we want to protect?

- Process memory
  - Passwords, personal information, execution pathways
- Files
  - User passwords are stored in a file. Who should be able to write?
- I/O devices
  - e.g. Who should be able to print?
  - e.g. Who should be able to send network messages?
- Resource decisions!
  - We need to make sure we are still the one allocating memory/compute

The OS already controls the resources we need to protect

That makes it the natural place to enforce security policy

Core Responsibilities

Because...

- Owns memory management & sharing
- Starts/pauses/terminates processes
- Makes resources available

Therefore...

- Protect each process's memory
- Make reasonable choices
- Allocate them safely

Security Goals

- **Confidentiality**: keep information secret
- **Integrity**: keep information correct
- **Availability**: keep the system up and running

Example: A cryptominer virus

- Mines crypto using your GPU, without your permission
  - Increases your power bill
  - Reduces your game’s FPS

What goals does this circumvent?

Availability

Security design is about tradeoffs

- The goal is not just to block attacks
- The goal is to protect the system without making it unusable
- Good security design reduces damage when something goes wrong

Design Principles

Keep it simple

- Economy of mechanism
- Open design

Enforce policy carefully

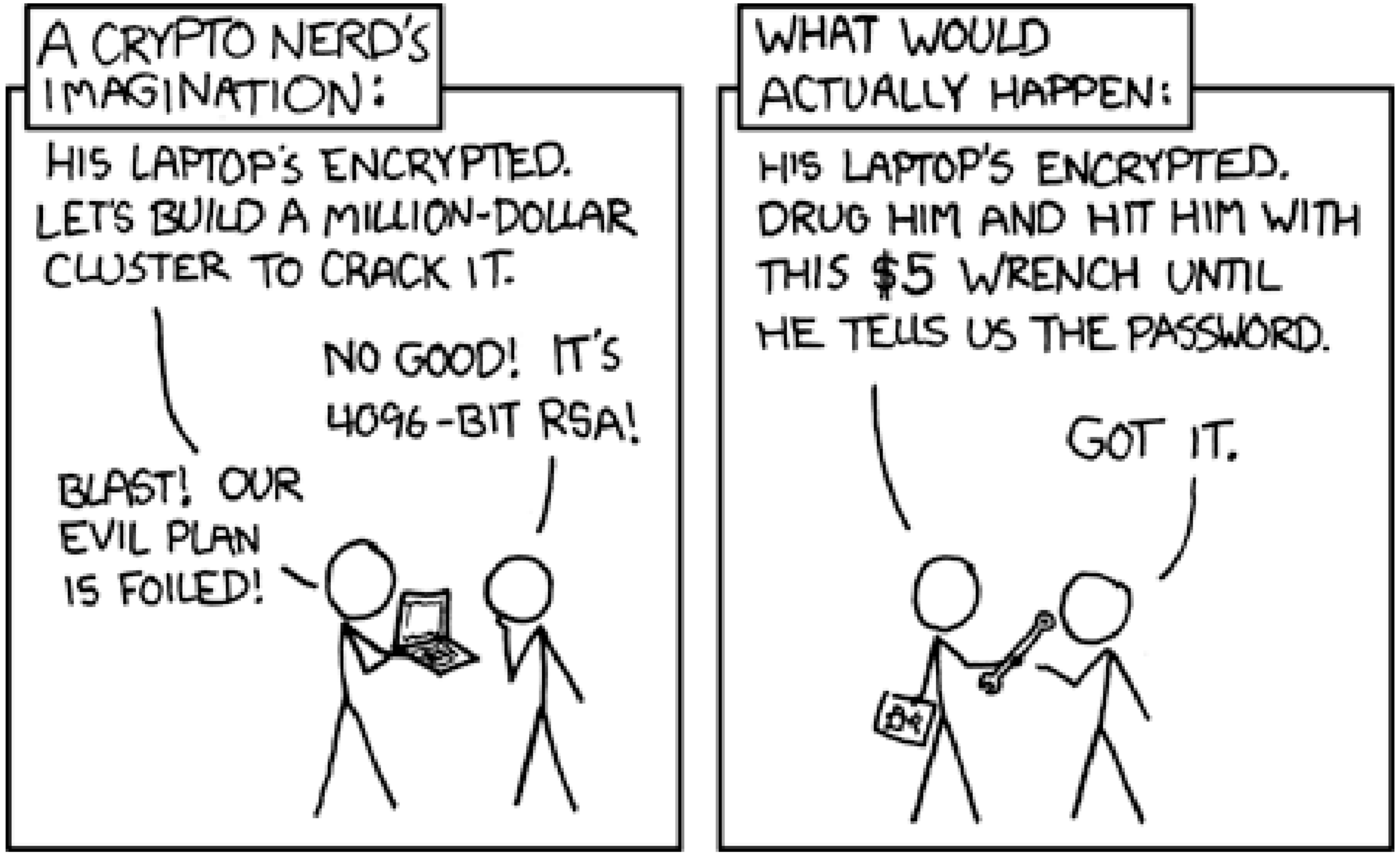
- Fail-safe defaults
- Complete mediation
- Separation of privilege

Limit blast radius

- Least privilege
- Least common mechanism
- Acceptability

Encouraging good security

- Weakest links will break first
  - Breaking a cryptographic hash is really hard
  - Breaking "password" (i.e. the literal word as a password) is really easy



System Calls - Not all are equal

- `getpid()`
- `getppid()`
- `time()`
- `wait()`
- `exit()`

- `fork()`
- `mmap()`
- `mprotect()`
- `socket()`
- `recv()`

Summary

Security is essential to any system

- Virtualization and sharing are useful, but they create policy decisions that the OS must enforce

The security goals are **confidentiality**, **integrity**, and **availability**

Good security comes from simple mechanisms, careful checks, and designs people can actually use

- In practice: focus on what works, and make sure it still works when things go wrong